

Department of Computer Science
University of Pretoria

Programming Languages
COS 333

Practical 3: Logic Programming

20 March, 2026

1 Objectives

This practical aims to achieve the following general learning objectives:

- To gain and consolidate some experience writing logic programs in Prolog;
- To consolidate the concepts covered in Chapter 16 of the prescribed textbook.

2 Plagiarism Policy

Plagiarism is a serious form of academic misconduct. It involves both appropriating someone else's work and passing it off as one's own work afterwards. Thus, you commit plagiarism when you present someone else's written or creative work (words, images, ideas, opinions, discoveries, artwork, music, recordings, computer-generated work, etc.) as your own. Note that using material produced in whole or part by an AI-based tool (such as ChatGPT) also constitutes plagiarism. Only hand in your own original work. Indicate precisely and accurately when you have used information provided by someone else. Referencing must be done in accordance with a recognised system. Indicate whether you have downloaded information from the Internet. For more details, visit the library's website: <http://www.library.up.ac.za/plagiarism/>.

3 Submission Instructions

Upload your practical-related source code file to the appropriate assignment upload slot on the ClickUP course page. You will be implementing all the practical's tasks in the same file. Name this file `u99999999.p1`, where `99999999` is your student number. Multiple uploads are allowed, but only the last one will be marked. The submission deadline is **Monday, 13 April 2026, at 12:00**.

4 Background Information

For this practical, you will be writing programs in SWI-Prolog version 9.0.4:

- Write all your Prolog programs (consisting of facts and rules) in a single source file. To do this, launch the SWI-Prolog interpreter in Windows, and select “New...” under the “File” menu. Type a file name, and click “Save”. An edit dialogue will then pop up, through which you can write your program source file. Source code files are saved by selecting “Save buffer” under the “File” menu. Note that SWI-Prolog is sensitive to end of line characters, so it is strongly recommended that you use this built in editor to write your programs. Also, be sure to place facts and rules on separate lines. Note that only Prolog predicate implementations are provided in your source file. You should not provide queries in your program source code.
- In order to test your implementation, select “Consult...” under the “File” menu, then choose the program source file you have written. You can then type in queries in the main SWI-Prolog window, which is in interactive mode.
- In interactive mode, a **true** response means that the interpreter can prove your query to be true, while a **false** response means that the interpreter cannot prove your query to be true. If you include variables in your query, the interpreter will respond with a value that makes the query true, or **false** if it cannot find such a value. Pressing **r** after a query response will re-query the interpreter (you can do this repeatedly, until Prolog runs out of responses), while pressing **Enter** will end the query.
- The course ClickUP page contains documentation related to SWI-Prolog [1], which contains detailed information on the operation of the SWI-Prolog interpreter, and details on the implementation of the Prolog programming language that SWI-Prolog provides.
- **Note that you may only use the simple constants, variables, list manipulation methods, and built-in predicates discussed in the textbook and slides. In particular, do not use if-then, if-then-else, and similar constructs. You may NOT use any more complex predicates provided by the Prolog system itself. In other words, you must write all your own propositions. Failure to observe this rule will result in all marks for a task being forfeited.**
- You may implement and use the propositions defined in the textbook and slides (e.g. `member`, `append`, and `reverse`). Note that you must provide the implementation for any of these propositions in your source file. Also note that there are some built-in propositions that correspond to the propositions defined in the textbook, which you may not use.
- You may implement helper propositions if you find them necessary.

5 Practical Tasks

For this practical, you will need to explore and implement logic programming concepts, including list processing. All of the following tasks should be implemented in a single source code file.

5.1 Task 1

Define a series of facts relating to computers and routers, such as the following:

```
computer(pc1).
computer(pc2).
computer(pc3).
computer(pc4).
computer(pc5).
computer(pc6).

router(router1).
router(router2).
router(router3).

connection(pc1, router1).
connection(router1, pc2).
connection(router1, router2).
connection(router3, router2).
connection(router2, pc3).
connection(pc4, router3).
connection(pc5, pc6).
```

Note that the **only** facts you are allowed to define are the `computer`, `router`, and `connection` propositions. If you define any facts other than these three propositions, you will **forfeit all marks for this task**. The `computer(X)` proposition means that `X` is a computer. The `router(X)` proposition means that `X` is a network router. The `connection(X, Y)` proposition means that `X` and `Y` are connected by a network connection. Also note the following:

- You may assume that no atom may be both a computer and a router.
- In the `connection(X, Y)` proposition, `X` and `Y` can both be either a computer or a router.
- Inverse connections are implied, but facts covering these inverse connections need not be provided. For example, in the provided code `connection(pc1, router1)` implies that `connection(router1, pc1)` also holds, even though the `connection(pc1, router1)` predicate is not explicitly included.

You must define the following propositions by means of rules that use the fact propositions listed above:

- The proposition `routersConnected(X, Y)`, which is true when the routers `X` and `Y` are connected to one another either directly or through a third router. For simplicity, assume that there are no more than three routers connected to each other (that is, routers are separated by at most one other router). Given the facts in the example above, `router1`, `router2`, and `router3` are all connected.
- The proposition `computerConnectedToRouter(X, Y)`, which is true when computer `X` is connected to router `Y`. Given the facts in the example above, `pc1` and `pc2` are both connected to `router1`, `pc3` is connected to `router2`, and `pc4` is connected to `router3`.
- The proposition `computersConnected(X, Y)`, which is true when computers `X` and `Y` are connected either directly or indirectly via routers. Given the facts in the example above, `pc5` and `pc6` are directly connected to one another. Additionally, `pc1`, `pc2`, `pc3`, and `pc4` are connected through `router1`, `router2`, and `router3`. It is suggested that you use the `computerConnectedToRouter` and `routersConnected` propositions in the rule (or rules) for the `computersConnected` proposition.

Note that the `routersConnected` and `computersConnected` propositions are both bidirectional. For example, both `routersConnected(router1, router3)` and `routersConnected(router3, router1)` should be true. Similarly, for example, both `computersConnected(pc1, pc4)` and `computersConnected(pc4, pc1)` should be true. Also note that you may define any additional helper propositions that you choose to, if they help in your implementation.

Hint 1: Be sure to test all your propositions thoroughly, including both propositions that should be true, as well as propositions that should be false. In many ways, excluding false propositions in your implementation can be more important than including true propositions. Including incorrect false propositions indicates that your implementation is too general.

Hint 2: To test whether invalid objects are included in your proposition, try entering queries involving variables, such as `computersConnected(X, Y)`. If you re-query repeatedly, this will list all objects for `X` and `Y` that satisfy the proposition.

5.2 Task 2

Write a Prolog proposition named `getValuesGreaterThan` that has three parameters. The first parameter is a numeric atom, while the second and third parameters are both simple numeric lists (i.e. lists containing only integers). The proposition defines the third parameter to be a list containing only the values contained in the second parameter list that are greater than the first parameter value, in the same order they appear in the second parameter list. To illustrate the use of the `getValuesGreaterThan` proposition, consider the following queries and responses:

```
?- getValuesGreaterThan(5, [], X).
X = [].

?- getValuesGreaterThan(5, [-1, 4], X).
X = [].

?- getValuesGreaterThan(5, [-1, 6, 0, 8, -5], X).
X = [6, 8].
```

Test the `getValuesGreaterThan` proposition using the provided example queries, as well as your own test input, and verify that the proposition works as you expect.

Hint: You may find that when you test your proposition on a list, you get the correct instantiation of the third parameter, but re-querying produces a **false** response. This usually happens because, on backtracking, a base case or recursive case is re-evaluated using an inappropriate recursive case, which fails. It isn't necessary to address this problem for this question. However, if you wish to avoid this behaviour, you can add a cut. A cut is a goal that is automatically satisfied, but does not allow backtracking past it. It can be used as follows:

```
proposition([]) :- !.
```

Assuming that the empty list denotes a base case, the `!` symbol is a cut. What this means is that once the base case of an empty list is reached, Prolog won't try to backtrack and find another way to satisfy the base case. This also works for recursive cases, where the cut is typically provided as the last sub-goal in the antecedent.

5.3 Task 3

Write a Prolog proposition named `countConsecutivePairs` that has two parameters. The first parameter is a simple list (i.e. a list containing only symbolic atoms or integers). The second parameter is an integer. The proposition is true if the second parameter represents the number of consecutive pairs of equivalent values in the list. A pair is considered a single unit, so the list `[atm, atm, atm]` contains one pair (the first two `atm` atoms), while the list `[atm, atm, atm, atm]` contains two pairs (the first two `atm` atoms, and the last two `atm` atoms). To illustrate the use of the `countConsecutivePairs` proposition, consider the following queries and responses:

```
?- countConsecutivePairs([], X).
X = 0.

?- countConsecutivePairs([a, b], X).
X = 0.

?- countConsecutivePairs([b, a, a, a, c], X).
X = 1.

?- countConsecutivePairs([b, b, a, c, c, c, d], X).
X = 2.
```

Test the `countConsecutivePairs` proposition using the provided example queries, as well as your own tests, and verify that the proposition works as you expect.

Hint: Refer to the slides for an example of how to represent a list containing at least two elements at the start of the list.

6 Marking

Each of the tasks will count 5 marks for a total of 15 marks. Submit both tasks implemented in the same source code file. Do not upload any additional files other than your source code. Both the implementation and the correct execution of the propositions will be taken into account. **You will receive zero for a task that uses a language feature you are not allowed to use.** Your program code will be assessed during the practical session in the week of **Monday, 13 April 2026**.

References

- [1] Jan Wielemaker. *SWI-Prolog Reference Manual*. University of Amsterdam, January 2018.